



Requirements Definition Document

Secure PubSub

In collaboration with:



26th October 2024

Secure PubSub.....	1
1. Introduction.....	3
1.1. Purpose.....	3
1.2. Scope.....	4
1.2.1. OPC UA Information Model.....	4
1.2.2. OPC UA PubSub architecture.....	5
1.2.2.1. Publisher Component.....	5
1.2.2.2. Subscriber Component.....	7
1.2.3. Security.....	9
1.3. Definitions, Acronyms, Abbreviations.....	10
1.4. Revision history and change log.....	11
1.5. Reference Documents.....	12
1.6. Document Structure.....	13
2. Specific Requirements.....	14
2.1. High-level Functional Requirements view.....	15
2.2. Technical Functional Requirements list.....	17
2.3. Mapping table between high-level and technical requirements.....	28
2.4. Non-Functional Requirements.....	29
2.5. User stories.....	30
2.5.1. Publisher.....	30
2.5.2. Subscriber.....	32
2.5.3. SKS.....	34
2.5.4. Certificate & Private key generation tool.....	36
2.5.5. Round-trip time measuring tool.....	37
2.6. Mapping table between user stories and requirements.....	39
2.7. Graphical User Interface.....	41

1. Introduction

1.1. Purpose

The customer for this project is ABB, a Swedish-Swiss multinational electrical engineering corporation and headquartered in Zurich. ABB was ranked 340th in the Fortune Global 500 list of 2020 and has been a global Fortune 500 company for 24 years.

The customer wants to improve the state of the existing OPC UA (Open Platform Communications Unified Architecture) technology stack, which is a machine-to-machine communication protocol for industrial automation developed by the OPC Foundation. It is designed for secure, reliable, and platform-independent exchange of data between devices and systems. The OPC UA specification is implemented by many vendors. This project will incorporate additional features into the open62541 open-source implementation.

The project's aim can be explained into three main points:

- Provide convenient and easy **configurability** of OPC UA nodes
- Enhance **security and reliability** in the standard OPC UA communication
- Supply a **measuring tool** to assess the performance of the system

These three points will be deeply clarified in the remainder of this document, offering to the reader also an overview of the open62541 technology to familiarise with the main topic of the project.

Disclaimer: It is important to mention that while the documentation for this project was designed to be as readable as possible for a non-technical reader, both the customer and the final users of the solution are people with a sound technical background on the technology employed, hence the need to descend into technicalities and details throughout the requirements elicitation and design process.

1.2. Scope

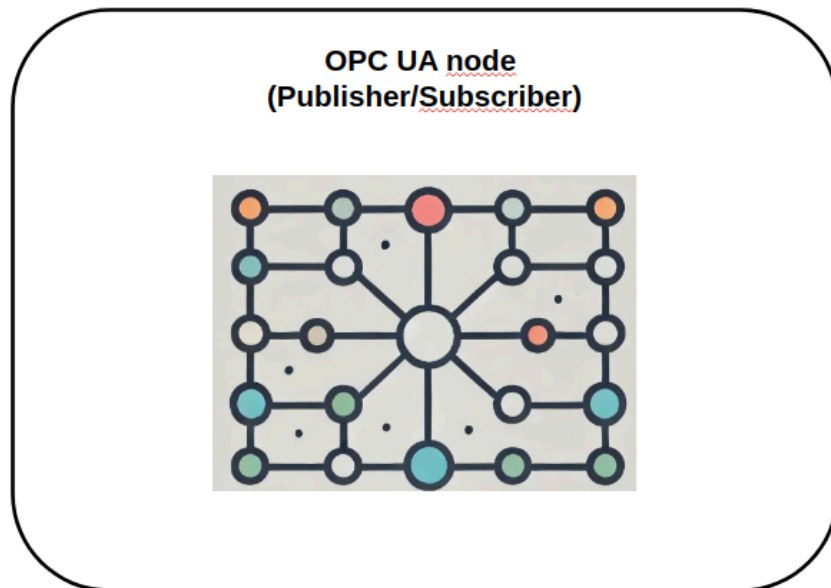
This project is intended for a **technical user** of the open62541 stack and as such it requires technical knowledge also at the requirements definition level to understand what must be done and achieved. Consequently, this section of the document will introduce the reader to the open62541 technology, highlighting only the relevant aspects and technicalities that are needed to understand the requirements definitions.

1.2.1. OPC UA Information Model

The main objective of OPC UA is to provide a secure, platform-independent, and standardised framework for data exchange in industrial automation and other industries, enabling seamless and reliable interoperability between different systems and devices.

This objective is achievable thanks to an abstraction in data representation called Information Model. The OPC UA Information Model enables a unified representation of all data and operations within a system. It provides a structured hierarchy where each element, known as a *node*, represents either data, a concept, or a functionality within the system. By adhering to this model, OPC UA promotes interoperability across diverse environments, allowing machines, sensors, and applications to communicate using a common language of data and operations.

The Information Model is fundamentally built upon Nodes. In OPC UA, a Node is a basic element that represents various forms of data or functions within the system. Nodes are organised within a hierarchy and are defined by unique identifiers, attributes, and references.



Nodes in the information model come in various types, each with specific characteristics:

- **Objects:** Represent entities within the system, such as devices, assets, or organisational units. Objects are often used to model complex entities and can contain variables, methods, and other nodes.

- **Variables:** Hold data values, such as real-time readings or static configurations. Variables can have data types ranging from simple (e.g., integers, strings) to complex (e.g., structured data types or custom objects).
- **Methods:** Represent executable functions within the model (callback functions), providing interfaces to interact with or control the system. For instance, a Method could represent an operation to start a machine or reset a sensor.
- **Data Types:** Define the format and constraints of data that can be represented within the model. OPC UA supports a wide range of standard data types.

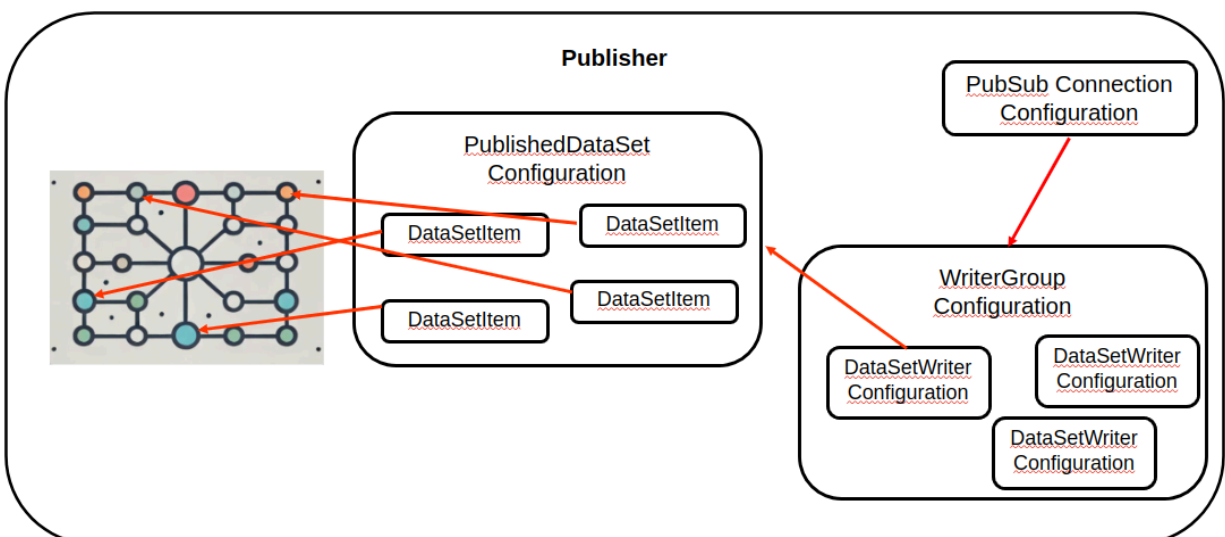
The Information Model's hierarchical design enables an organised approach to data structuring, where nodes are connected and contextualised through References. For instance, an Object representing a machine might have Variables for each sensor reading, Methods for control operations, and References linking it to other relevant system components. The model's structure allows users to navigate the hierarchy, facilitating efficient access to data and functionality.

1.2.2. OPC UA PubSub architecture

The OPC UA PubSub architecture is one of the communications models offered by the OPC UA standard. It enables efficient and structured data exchange between distributed systems, by using a Publisher-Subscriber model. In open62541, this model is implemented with precise structures and configurations, allowing nodes within an OPC UA information model to be published and read in real time by subscribers. In particular, this project focuses on a brokerless implementation of the PubSub model, in which data exchange is supported through a UDP multicast connection. Below is a breakdown of the architecture, focusing on the Publisher and Subscriber components and their respective internal data structures and configurations.

1.2.2.1. Publisher Component

In the OPC UA PubSub framework, the Publisher is responsible for broadcasting data across the network, serving as the source of information within the system. Each Publisher is configured to establish a Connection to the communication network (UDP multicast in this project), defining essential transport parameters to manage how data is packaged and transmitted.



As it is possible to notice from the illustration, the number of configuration parameters that are needed to setup a publisher machine is noticeable, and this leads to the justification of why it is relevant to extract the configuration data out of the code (the programmatic approach) and provide a text-based representation, which would facilitate the developer's work.

Going in-depth on the elements that are needed on the publisher side, we start analysing the structure that is offered by the open62541 stack for what concerns the creation of a connection. The *UA_PubSubConnectionConfig struct* allows to setup parameters like:

name : UA_String	Name of the connection.
enabled : UA_Boolean	State of the connection.
nodeId : UA_PublisherId	Identifier for the publisher node inside the open62514 memory space.
address : UA_Variant	(UDP multicast) address (and port number) on which data is periodically published.

Another central component within the Publisher is the *PublishedDataSet*. The *PublishedDataSet* is a logical collection of *DataSetFields*, each representing a node in the OPC UA information model. This design allows specific variables or data points within the information model to be organised and broadcasted in a structured manner. Each *DataSetField* in the *PublishedDataSet* links directly to a unique node within the OPC UA information model, enabling the Publisher to retrieve real-time values from these nodes for transmission. The *UA_PublishedDataSetConfig struct* allows a user of the open62541 stack to specify the configurations for a specific *PublishedDataSet*, in particular the most relevant fields are:

name : UA_String	Name of the <i>PublishedDataSet</i> .
publishedDataSetType : UA_PublishedDataSetType	Type of data published (variables, events...)

As for the *DataSetFields*, they can be thought of as pointers to specific nodes of the information model of the publisher. Some relevant fields for this structure in the open62541 stack are:

publishedVariable : UA_NodeId	Pointer to the specific node to be published (in the information space of the publisher).
attributId : UA_UInt32	Specific field within the node to be published (for instance its internal value, its qualified name...).

The *WriterGroup* is another component that is responsible for managing and structuring the flow of data from the Publisher to the network. Each *WriterGroup* operates on a specific publishing interval, defining how frequently data is sent out. The *WriterGroup* is also responsible for creating *NetworkMessages*,

which encapsulate the data from one or more DataSetWriters and apply consistent encoding and security configurations.

All relevant configurations parameters for a WriterGroup can be stored in a *UA_WriterGroupConfig struct* offered by the open62541 stack:

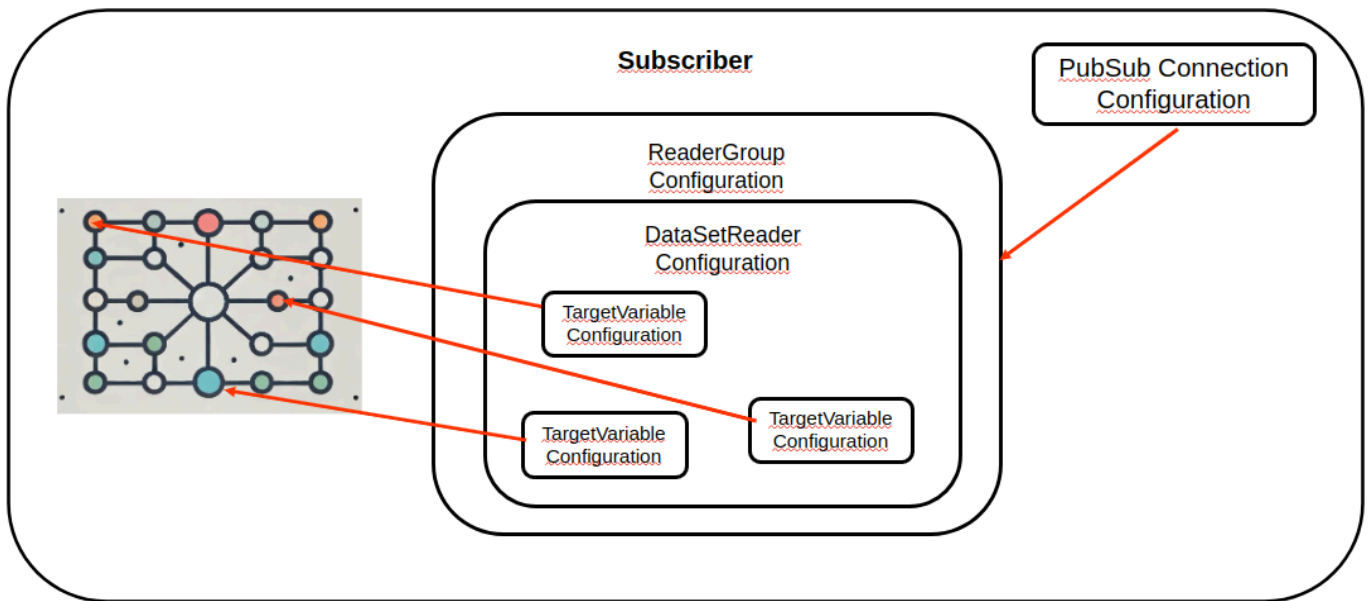
name : UA_String	Name of the WriterGroup.
Enabled : UA_Boolean	State of the WriterGroup.
writerGroupId : UA_UInt16	Identifier for the WriterGroup.
publishingInterval : UA_Duration	The time interval at which the group collects and sends data messages to the network.

Within each WriterGroup, there are one or more DataSetWriters. Each DataSetWriter is linked to a single PublishedDataSet. The DataSetWriter generates DataSetMessages from its assigned PublishedDataSet, formatting the data into a message structure that the Subscriber can interpret. The DataSetWriter's configuration, such as the DataSetFieldContentMask and security settings, governs how the PublishedDataSet's fields are encoded and transmitted within each NetworkMessage. Some relevant fields for a DataSetWriter can be configured in the *UA_DataSetWriterConfig struct* offered by the open62541 stack:

name : UA_String	Name of the DataSetWriter.
dataSetWriterId : UA_UInt16	Identifier for the DataSetWriter.
keyFrameCount : UA_UInt32	How often a complete data set (a key frame) is sent as opposed to only sending delta frames (incremental changes).
publishingInterval : UA_Duration	The time interval at which the group collects and sends data messages to the network.

1.2.2.2. Subscriber Component

The Subscriber component in OPC UA PubSub is the entity that receives data from Publishers and maps this data to its local information model. The Subscriber listens for incoming messages based on a configured Connection to the Publisher's network. This connection must align with the Publisher's settings, ensuring the compatibility required to interpret the Publisher's NetworkMessages.



Similar to the Publisher, the Subscriber's Connection configuration specifies the transport protocol and network settings necessary to receive data. It includes parameters such as the multicast address and protocol specifications. Proper alignment with the Publisher's connection parameters is crucial for the Subscriber to successfully receive and process the published data. The parameters to establish a connection have been detailed in the previous section.

The Subscriber organises its data reception and interpretation through a *ReaderGroup*. A ReaderGroup can be configured in the open62541 stack thanks to the *UA_ReaderGroupConfig struct*. Some relevant fields for this project are:

name : UA_String	Name of the ReaderGroup.
------------------	--------------------------

Each ReaderGroup manages one or more *DataSetReaders*, which are responsible for interpreting specific incoming data messages. Each DataSetReader in the ReaderGroup is designed to interpret and parse messages from a specific DataSetWriter (on the publisher side). The DataSetReader is configured through a *UA_DataSetReaderConfig struct* in the open62541 stack. Some relevant fields are:

name : UA_String	Name of the DataSetReader.
publisherId : UA_PublisherId	Identifier of the publisher from which data is expected.
writerGroupId : UA_UInt16	Identifier of the WriterGroup from which data is expected.
dataSetWriterId : UA_UInt16	Identifier of the DataSetWriter from which data is expected.
dataSetMetaData : UA_DataSetMetaData	Template representing the expected

structure of the data coming in from the publisher (specifies data types and other parameters).

The last entry in the table, `UA_DataSetMetaDataType`, is itself a complex data structure that is able to encapsulate a template of the data set that the subscriber is expecting from the publisher. In particular, the `UA_DataSetMetaDataType` contains a set of `UA_FieldMetaData`, which are basically data type descriptors. Any deviation between the incoming `DataSetMessage` and the `DataSetMetaData` may lead to message rejection or error handling by the Subscriber, thereby preserving data integrity.

Finally, the `DataSetReader`'s configuration includes target variables, which are pointers to local nodes within the Subscriber's information model. Each field in the `DataSetMetaData` corresponds to a target variable, dictating where the data will be stored once received. This mapping allows the Subscriber to update specific nodes in its own information model based on the incoming values from the Publisher. Now, to configure these target variables in the open62514 stack, it is necessary to use the `UA_FieldTargetVariable struct`, which contains some important parameters like the `targetNodeId` (pointing to the local node where the subscriber wants to map the data coming from the publisher).

1.2.3. Security

The PubSub architecture is brokerless, with the network itself serving as the middleware for this solution - this is possible through UDP Multicast. The primary vulnerabilities of this solution are as follows:

- **Confidentiality:** Any entity that joins the UDP Multicast group can intercept and exploit the published data.
- **Authentication:** Any entity can inject data into the UDP Multicast group by impersonating a legitimate publisher.

The solution adopted to address these aspects is based on the following design: the published data is encrypted by the Publisher using a symmetric key, which is also used by the Subscriber to decrypt it.

To achieve this solution while maintaining the implicit nature of a publish-subscribe communication model, a Security Key Server (SKS) has been designed to handle the symmetric key logic, including its generation, rotation, and distribution among all components deployed in the network. This is just another OPC UA node that acts differently from a Publisher and a Subscriber and it is instructed only to take care of security-related aspects.

To enhance scalability, the SKS is designed to manage multiple groups of Publishers and Subscribers that communicate in an isolated manner. This isolation is achieved through the management of distinct symmetric key groups rather than relying on separate UDP Multicast addresses. As a result, multiple groups of Publishers and Subscribers can share the same UDP Multicast address while maintaining secure and isolated communication through the use of unique symmetric keys for each group. These groups are called Security Groups.

2.1.2.2. Security Group

The Security Group is an abstraction representing a group of symmetric keys used within the PubSub architecture to enable secure communication. Each Security Group is identified by a unique ID, and all

Publishers and Subscribers within the same Security Group share the same symmetric keys, allowing them to communicate securely.

1.3. Definitions, Acronyms, Abbreviations

OPC UA	Open Platform Communications Unified Architecture
Publisher	Component that sends data to subscribers in the Pub/Sub model.
Subscriber	Component that receives data published by publishers.
DataSetWriter	Configures the structure and content of data to be sent by a publisher.
DataSetReader	Configures how to read and process incoming data from a publisher.
Writer Group	Groups multiple DataSetWriters for coordinated data transmission.
Reader Group	Groups multiple DataSetReaders to read published data collectively.
PublishedDataSet	Selects data to be published by a DataSetWriter on a publisher node.
Metadata	Information needed by subscribers to correctly interpret incoming data.
Secure Key Server (SKS)	Central component that distributes keys for securing communications in the Pub/Sub model.
Address Space	Logical representation of all nodes and relationships in the OPC UA model.
Node	Basic element of the OPC UA address space, representing data or functions.
PK	Private Key

1.4. Revision history and change log

Version 1 - 8.11.2024

Version 2 - 1.12.2024:

This version focuses on updating the requirements

- The Database integration with Publisher and Subscriber is no longer required.
- A certificate trust-list mechanism has been added to ensure the security of the key distribution channel

Version 3 - 15.12.2024

This version focuses on a refactoring over the previous ones for readability purposes:

- Better description of open62541 stack to give the reader more context
- Division of the requirements in two different views
 - High-level requirements: easy to understand by any consumer of the document, with or without technical knowledge of the underlying open62541 stack
 - Technical requirements: more fine-grained view of the same requirements in which all the technical aspects are highlighted, this section is intended for a technical reader, with background knowledge of the open62541 stack.
- Mapping table between high-level and technical requirements
- Removal of old requirements on certificate generation inside the publisher and subscriber nodes (ex PUB-6 and SUB-5)

1.5. Reference Documents

UPC UA reference <https://reference.opcfoundation.org/>

Open62541 reference <https://www.open62541.org/doc/v1.4.0/>

1.6. Document Structure

The requirements document for the Secure PubSub project is structured as follows:

Chapter 1, Introduction, provides an overview of the document's purpose, the scope of the project, and background information. It includes details like the definitions, acronyms, and abbreviations used throughout the document. The Revision history section captures changes made to the document over time, while Reference Documents lists any external documents referenced. The Document Structure subsection outlines the organisation of the content.

Chapter 2, Specific Requirements, is split into two sections.

The first part provides a high-level view of the requirements for this project, in which most technicalities have been abstracted away to make this paper more readable for users without a solid background knowledge on the OPC UA technology.

The second part contains the detailed requirements for the Secure PubSub system, incorporating all technicalities and technology-related aspects. It starts with the Requirements list, which enumerates all the functional and non-functional requirements of the system. The User stories section describes typical user interactions and use cases for components like the Publisher, Subscriber, Secure Key Server (SKS), and a Round-trip time measuring tool. Lastly, the Mapping table between user stories and requirements provides a correlation between the defined user stories and their respective requirements, ensuring traceability.

At the very end, a little explanation is provided to the reader on the reasons why a GUI is not provided for this project.

2. Specific Requirements

This section of the document is divided into two parts.

The first paragraph *High-level Functional Requirements view* is provided for non-technical readers, more specifically for consumers of this document without a solid background on the open62541 technology. It shows the main achievements that have to be accomplished during the development of this software solution at a high-level.

The second paragraph *Technical Functional Requirements list* breaks down the high-level requirements into fine-grained technical requirements that have to be completed by this project.

The decision to provide two distinct views of the requirements document stems from the fact that both the customer and the end-user of the product are experts in the OPC UA standard and the open62541 stack. By offering these two views, we aim to present a high-level perspective for readers who may lack specific technical expertise, while simultaneously ensuring that the detailed technical requirements meet the advanced knowledge of the stakeholders.

2.1. High-level Functional Requirements view

The following table shows a high-level view of the requirements for this project, without entering into the technical details.

ID	Name	Description
PUB-0.1	External Publisher Configuration	The system shall allow the user to configure an OPC UA publisher node through an XML configuration file.
PUB-0.2	Publisher Certificate trust-list	The system shall allow the user to configure a certificate trust-list on an OPC UA publisher node to enforce secure communication between components.
SUB-0.1	External Subscriber Configuration	The system shall allow the user to configure an OPC UA subscriber node through an XML configuration file.
SUB-0.2	Subscriber Certificate trust-list	The system shall allow the user to configure a certificate trust-list on an OPC UA subscriber node to enforce secure communication between components.
SKS-0.1	External SKS Configuration	The system shall allow the user to configure an OPC UA SKS node through an XML configuration file.
SKS-0.2	SKS database integration	The Secure Key Server (SKS) shall store encryption keys and security configurations in a database (and not in volatile memory).
SKS-0.3	SKS Certificate trust-list	The system shall allow the user to configure a certificate trust-list on an OPC UA SKS node to enforce secure communication between components.

ID	Name	Description
RTT-0.1	Performance Monitoring Tool	The system shall implement a tool to configure the deployment of a simple network of OPC UA nodes (with and without the SKS) to measure the performance of the final product of this project.
RTT-0.2	Performance Monitoring Tool Configuration	The RTT Performance Monitoring Tool shall allow the user to configure the deployment of the simple network of OPC UA nodes in a text-based manner.
RTT-0.3	Performance Monitoring Tool Collection of Results	The RTT Performance Monitoring Tool shall log the results of the monitoring process in a structured format (CSV).
COM-0.1	Communication of OPC UA nodes	The system shall allow the user to configure (in a text-based fashion) and start all OPC UA nodes in such a way that they are able to correctly communicate and exchange data over a network, with the security layer of the SKS enabled.

2.2. Technical Functional Requirements list

In the following table, the high-level requirements are broken down into more technical details. The description is divided into three tables, one for each type of OPC UA node (Publisher, Subscriber, SKS).

Publisher configuration				
ID	Name	Description	Source	Motivation
PUB-1	Publisher Network configuration	The system shall allow the user to configure the publisher's network configuration field through a text-based configuration file	System design and Customer Requirement	To ensure proper network configuration for the publisher node.
PUB-1.1	Network interface IP address	The system shall allow the user to define the IP address of the network interface used on which the publisher server will operate through a text-based configuration file	Customer Requirement	To ensure that the correct network interface is used for communication.
PUB-1.2	Network Server Port	The system shall allow the user to define the Port number on which the publisher server will accept client communications	Customer Requirement	To ensure proper client-server communication on the right port.
PUB-1.3	Multicast IP and Port	The system shall allow the user to define the multicast IP address and port as a destination to publish data using a text-based configuration file	Customer Requirement	To enable multicast communication for efficient data distribution.
PUB-1.4	OPC UA node ID	The system shall allow the user to define the node id through a text-based configuration file	OPC UA specification	Mandatory field to allow communication between OPC UA nodes
PUB-1.5	Enable Connection	The system shall allow the user to enable or disable the publisher network connection through a text-based configuration file	Customer Requirement	To allow dynamic control over the publisher's network connection.
PUB-1.6	Connection name	The system shall allow the user to define the connection name	Customer Requirement	To uniquely identify and manage

		through a text-based configuration file		publisher connections.
PUB-2	Published Dataset configuration	The system shall allow the user to configure a published dataset through a text-based configuration file	System design	To manage and configure datasets that are being published.
PUB-2.1	Published Dataset name	The system shall allow the user to configure the name of a published dataset through a text-based configuration file	Customer Requirement	To allow identification and management of datasets being published.
PUB-3	Published Dataset Field	The system shall allow the user to configure a published datasetField through a text-based configuration file	System design	To ensure flexibility in configuring fields of the published dataset.
PUB-3.1	Dataset Field Node id	The system shall allow the user to configure the node id of the data element hosted on the server to be published through a text-based configuration file	OPC UA specification	Mandatory field to ensure proper mapping and identification of dataset fields.
PUB-3.2	Dataset Field Namespace Index	The system shall allow the user to configure the namespace index of the data element hosted on the server to be published through a text-based configuration file	OPC UA specification	Mandatory field to define the namespace index for proper data mapping and referencing.
PUB-3.3	Dataset Field Attribute id	The system shall allow the user to configure the attribute id of a dataset field through a text-based configuration file	OPC UA specification	Mandatory field to allow users to configure the specific attribute of a dataset field to publish.
PUB-4	Writer Group configuration	The system shall allow the user to configure a writer group to be a part of a certain security group through a text-based config	Security requirement and Customer Requirement	To ensure the correct configuration of writer groups for publishing and setup security features.
PUB-4.1	Writer Group name	The system shall allow the user to configure the name of a Writer Group through a text-based config	Customer Requirement	To identify and configure the writer group for

				data publication.
PUB-4.2	Writer Group enable flag	The system shall allow the user to configure the status of a Writer Group (enabled or disabled) through a text-based config	Customer Requirement	To allow dynamic control over whether the writer group is active or not.
PUB-4.3	Writer Group numeric identifier	The system shall allow the user to configure the numeric identifier of a Writer Group through a text-based config	System design	Mandatory field to provide a unique identifier for the writer group.
PUB-4.4	Writer Group publishing interval	The system shall allow the user to configure the publishing interval (for periodic publishing of data) of a Writer Group through text-based config	Customer Requirement	To allow configuration of periodic data publishing for the writer group.
PUB-5	Dataset Writer configuration	The system shall allow the user to configure a dataset writer to publish a certain dataset type through a configuration file	Customer Requirement	To provide configuration options for dataset writers and their associated datasets.
PUB-5.1	Dataset Writer name	The system shall allow the user to configure the dataset writer name through a configuration file	Customer Requirement	To enable identification of the dataset writer for configuration and management.
PUB-5.2	Dataset Writer numeric id	The system shall allow the user to configure the dataset writer integer id through a configuration file	System design	Mandatory field to assign a unique numeric ID for the dataset writer.
PUB-5.3	Dataset Writer key frame number	The system shall allow the user to configure the dataset writer number of key frames through a configuration file	Customer Requirement	To control how many key frames are stored or transmitted by the dataset writer.
PUB-5.4	Dataset Writer Published Dataset Name	The system shall allow the user to configure the name of the published dataset linked to this dataset writer (see PUB-2.1)	System design	To associate a dataset writer with a specific dataset for

				publishing.
PUB-6	Certificate Management	The Publisher shall implement a certificate trust list to ensure secure channel communication with SKS	Security Standard and Customer Requirement	To ensure trusted and secure communication with SKS using certificates.
PUB-6.1	Certificate trust list configuration	The system shall allow the user to configure the directory path in which the certificates will be stored through a text-based configuration file	Security Standard and Customer Requirement	To manage where trusted certificates are stored for verification.
PUB-6.2	Load the certificate trust list	The system shall load and configure the certificates from the specified directory path	Security Standard	To implement and load the trusted certificates from the directory for secure communication.

Subscriber configuration				
ID	Name	Description	Source	Motivation
SUB-1	Subscriber Network configuration	The system shall allow the user to configure the subscriber's network for receiving data through a text-based configuration file	System design and Customer Requirement	To ensure the correct network configuration for receiving data on the subscriber side.
SUB-1.1	Network interface IP address	The system shall allow the user to define the IP address of the network interface used on which the subscriber server will operate through a text-based configuration file	Customer Requirement	To configure the specific network interface for the subscriber node.
SUB-1.2	Network Server Port	The system shall allow the user to define the Port number on which the subscriber server will accept client communications	Customer Requirement	To configure the correct port for subscriber-client communication.
SUB-1.3	Multicast IP and Port	The system shall allow the user to define the multicast IP address and	Customer Requirement	To enable multicast

		Port through which subscribe using a text-based configuration file		communication for efficient data reception.
SUB-1.4	OPC UA node ID	The system shall allow the user to define the node id through a text-based configuration file	OPC UA specification	To support integration with OPC UA for communication with the publisher.
SUB-1.5	Enable Connection	The system shall allow the user to enable or disable the subscriber network connection through a text-based configuration file	Customer Requirement	To provide the ability to manage the subscriber network connection dynamically.
SUB-1.6	Connection name	The system shall allow the user to define the connection name through a text-based configuration file	Customer Requirement	To uniquely identify the subscriber connection for management purposes.
SUB-2	Reader Group configuration	The system shall allow the user to configure a reader group through a text-based configuration file	Customer Requirement	To configure a reader group for subscribing to datasets efficiently.
SUB-2.1	Reader Group name	The system shall allow the user to configure the reader group name through a text-based configuration file	Customer Requirement	To allow proper identification and management of reader groups.
SUB-3	Dataset Reader configuration	The system shall allow the user to configure a dataset reader to receive a certain dataset from a specific dataset writer through a text-based configuration file	Customer Requirement	To define how datasets are received from a specific dataset writer.
SUB-3.1	Dataset Reader name	The system shall allow the user to define the name of the Dataset Reader through a text-based configuration file	Customer Requirement	To identify and configure the dataset reader.

SUB-3.2	Dataset Reader Language	The system shall allow the user to define the language of the Dataset Reader through a text-based configuration file	Customer Requirement	To configure the language for the dataset reader to ensure compatibility with the data.
SUB-3.3	Dataset Reader namespace index	The system shall allow the user to define the namespace index of the folder that will contain all received dataset fields on the subscriber server through a text-based configuration file	Customer Requirement	To ensure proper organization and referencing of received data fields.
SUB-3.4	Dataset Reader identifier of corresponding Publisher	The system shall allow the user to define the identifier of the publisher node from which data is expected, through a text-based configuration file	Customer Requirement	To associate the dataset reader with the specific publisher node.
SUB-3.5	Identifier of corresponding Writer Group	The system shall allow the user to define the identifier of the Writer Group from which data is expected, through a text-based configuration file	Customer requirement	To link the reader group with the corresponding writer group for data reception.
SUB-3.6	Identifier of corresponding Dataset Writer	The system shall allow the user to define the identifier of the Dataset Writer from which data is expected, through a text-based configuration file	Customer Requirement	To define the dataset writer from which the subscriber will receive data.
SUB-3.7	Dataset Reader Field configuration	The system shall allow the user to configure all the dataset reader fields through a text-based configuration file, where the Dataset Reader will map the incoming information from the publisher(s)	Customer Requirement	To properly map the fields from incoming datasets to the subscriber's data model.
SUB-3.7.1	Field name	The system shall allow the user to configure the field name through a text-based configuration file	Customer Requirement	To configure the field names for the dataset reader to match incoming data fields.
SUB-3.7.2	Field Namespace type	The system shall allow the user to configure the field type defined in namespace context through a text-based configuration file	Customer Requirement	To ensure the correct namespace context for the data fields in the dataset reader.

SUB-3.7.3	Field built-in type	The system shall allow the user to configure the field type defined in the built-in context through a text-based configuration file	Customer Requirement	To define the field types used for incoming data in the built-in context.
SUB-3.7.4	Field variable rank	The system shall allow the user to configure the field variable rank through a text-based configuration file	Customer Requirement	To configure the rank (array size) of the field for proper data interpretation.
SUB-4	Subscriber SKS reference	The system shall allow the user to configure the SKS server used to pull the keys through a text-based config	Customer Requirement	To enable secure communication by retrieving keys from the SKS server.
SUB-4.1	Subscriber SKS IP	The system shall allow the user to configure the IP address of the SKS server through a text-based config	Security Requirement	To define the correct IP address for the SKS server to allow key retrieval.
SUB-4.2	Subscriber SKS Port	The system shall allow the user to configure the port number of the SKS through a text-based config	Security Requirement	To configure the port for secure communication with the SKS server.
SUB-4.3	Subscriber SKS keyTimeInterval	The system shall allow the user to configure the time interval between two pulls of keys from the SKS through a text-based config	Security Requirement	To define the interval for securely pulling keys to maintain communication integrity.
SUB-4.4	Subscriber SKS Private Key Directory	The system shall allow the user to define the name of the directory in which private keys are stored on the SKS through a text-based config	Security Requirement	To securely manage the private keys used for communication.
SUB-5	Certificate management	The Subscriber shall implement a certificate trust list to ensure secure channel communication with SKS	Security Requirement	To ensure secure communication channels with the SKS through proper certificate management.
SUB-5.1	Certificate trust list configuration	The system shall allow the user to configure the directory path in which the certificates are stored through a	Security Requirement & Customer	To manage the certificates in a trusted directory

		text-based configuration file	Requirement	path for verification and security.
SUB-5.2	Load the certificate trust list	The system shall load and configure the certificates from the specified directory path	Security Requirement	To load and apply the trusted certificates from the specified directory for secure communication.

Secure Key Service configuration				
ID	Name	Description	Source	Motivation
SKS-1	SKS Network configuration	The system shall allow the user to configure the internet interface that the SKS will use through a text-based configuration file	Customer Requirement	To ensure the SKS has a properly configured network interface for communication.
SKS-1.1	Server Port	The system shall allow the user to configure the port used to accept requests by the SKS through a text-based configuration file	Customer Requirement	To configure the port for receiving requests by the SKS.
SKS-2	SKS Publisher configuration	The system shall allow the user to register publisher endpoints where the SKS will push keys through a text-based configuration file	Customer Requirement	To configure and register the endpoints where keys will be pushed by the SKS.
SKS-3	Security group key management configuration	The system shall allow the user to configure the key management properties for each security group on the SKS through a text-based configuration file.	Security Standard and Customer Requirement	To manage key configurations for each security group in SKS.

SKS-3.1	Security Group ID	The system shall allow the user to configure the Security Group ID.	Customer Requirement	To uniquely identify and manage the security group for key management .
SKS-3.2	Key Life Time	The system shall allow the user to configure Key Life Time.	Customer Requirement	To define the duration of the security keys before they expire.
SKS-3.3	Key Size	The system shall allow the user to configure the Key Size.	Customer Requirement	To configure the size of the keys for appropriate security levels.
SKS-3.4	Max Future Key Count	The system shall allow the user to configure the number of futures keys to generate.	Customer Requirement	To configure the number of future keys available for secure communication.
SKS-3.5	Max Past Key Count	The system shall allow the user to configure the number of past keys to store.	Customer Requirement	To configure the number of past keys retained for security and backup purposes.
SKS-3.6	Security Policy Uri	The system shall allow the user to configure the Policy Uri	Customer Requirement	To associate the security group with the appropriate security policy URL.
SKS-4	Subscriber Certificate management	The Sks shall implement a certificate trust list to ensure secure channel communication with Subscriber	Security requirement	To implement a trusted certificate management system for secure communication with the subscriber.
SKS-4.1	Certificate trust list configuration	The system shall allow the user to configure the directory path in which the Subscriber certificates are stored through a text-based configuration file	Security requirement and Customer Requirement	To define the directory path for storing the subscriber certificates.

SKS-4.2	Load the certificate trust list	The system shall load and configure the certificates from the specified directory path	Security requirement	To load and manage certificates for secure communication with the subscriber.
SKS-5	Publisher Certificate management	The Sks shall implement a certificate trust list to ensure secure channel communication with Publisher	Security requirement	To ensure secure communication with the publisher by implementing a trusted certificate management system.
SKS-5.1	Certificate trust list configuration	The system shall allow the user to configure the directory path in which the Publisher certificates are stored through a text-based configuration file	Security requirement	To define the directory path for storing the publisher certificates.
SKS-5.2	Load the certificate trust list	The system shall load and configure the certificates from the specified directory path	Security requirement and Customer Requirement	To load and configure the publisher certificates for secure communication.
SKS-6	Database storage for security keys	The SKS shall be able to store keys and retrieve them from a database to ensure persistence	System design and Customer Requirement	To ensure that security keys are stored persistently and can be retrieved when necessary.
SKS-7	Recovery mechanism	After a reboot, the SKS shall be able to check the database or create and configure it if needed, and populate it for each security group.	System design and Customer Requirement	To maintain continuity of key management by recovering or creating the database after a reboot.

Round-trip time measuring tool				
ID	Name	Description	Source	Justification
RTT-1	Network configuration for primary nodes	A user shall be able to configure the internet interface that the primary node will use through a text-based configuration file	Customer Requirement	To be able to deploy the system on any node with custom network settings
RTT-2	Network configuration for intermediate nodes	A user shall be able to configure the internet interface that the intermediate node will use through a text-based configuration file	Customer Requirement	To be able to deploy the system on any node with custom network settings
RTT-3	RTT rounds config	A user shall be able to specify how many rounds of RTT measurements will be run	Customer Requirement	To be able to specify how many rounds of the experiment should be done, and (by extension), how many data points will be collected
RTT-4	SKS customization	A user shall be able to specify whether the tool will include the SKS when running the experiments or not	Customer Requirement	To be able to detect performance difference (if any) between the secure and unsecure version of the protocol
RTT-5	Data collection	Data at the end of the experiment shall be stored	Customer Requirement	To be able to analyze the data post-experiment

2.3. Mapping table between high-level and technical requirements

High-Level Requirement ID	Low-Level Requirement IDs
PUB-0.1	PUB-1, PUB-2, PUB-3, PUB-4, PUB-5, PUB-6
PUB-0.2	PUB-7
SUB-0.1	SUB-1, SUB-2, SUB-3, SUB-4, SUB-5
SUB-0.2	SUB-6
SKS-0.1	SKS-1, SKS-2, SKS-3
SKS-0.2	SKS-6, SKS-7
SKS-0.3	SKS-4, SKS-5
RTT-0.1	RTT-1, RTT-2, RTT-3, RTT-4, RTT-5
RTT-0.2	RTT-1, RTT-2, RTT-3, RTT-4
RTT-0.3	RTT-5

2.4. Non-Functional Requirements

ID	Name	Description	Validation
NF-1	Usability of text-based configuration	The support for an external text-based configuration for the OPC UA nodes (Publishers, Subscribers, SKS) shall improve the usability of the OPC UA stack and shall be easy-to-understand for a new user	The text-based configuration has been designed to use names for all fields that are compliant with the OPC UA stack implementation and intuitive to understand. Validation of this requirement is going to be carried out through customer and users' feedback while employing the technology.
NF-2	Extensibility of the parser design	The parser component implementation shall be extensible to accommodate for additional fields that might be needed by a user of the technology	The parser implementation has been designed in a modular way, so that new parts can be easily integrated by following a simple pattern that is described in the Design Document for this project and is easy to elicit from the code implementation itself.
NF-3	Security of communication between OPC UA nodes	The overall cluster of OPC UA nodes deployed using the solution provided by this project shall support and enforce secure communication, with particular focus on confidentiality and integrity of data transmission.	The solution offered by the project fully integrates with the pre-existing SKS implementation and allows the configuration of all security-related aspects of OPC UA nodes in a text-based fashion. Without the correct setup of certificates and security groups, no communication will be enabled by the system.
NF-4	Dependability of SKS implementation	The SKS implementation shall be made dependable in order to continue the communication between publishers and subscribers even in the case of a short period of downtime for the SKS node. The SKS can crash, but it is equipped with persistent storage that allows it to recover and let the communication between publishers and subscribers continue without any downtime.	Multiple different clusters of OPC UA nodes will be deployed with different configurations and the recovery capabilities of the SKS will be tested by shutting down and restarting the SKS repetitively and verifying that all publisher-subscriber communications are able to continue without any loss of data. More details on the Acceptance Testing documentation for this project

2.5. User stories

2.5.1. Publisher

Id: PUBS-1

Title: Publisher network configuration

Description:

Before starting the Publisher:

- The user can specify in a config file a set of network configuration like:
 - The IP of the network interface used to publish
 - The port of the publisher server
 - The UDP multicast IP and Port
 - The Publisher id value
 - The Option to enable the connection
 - The Connection name
- The system parses the file
- If the parsing is unsuccessful, the program terminates and displays an error message
- Otherwise, the program continues to the next configuration step or starts to run the publisher if there aren't more configurations

Id: PUBS-2

Title: Publisher dataset configuration

Description:

Before starting the Publisher

- The user can specify in a config file the name of the publisher dataset
- The program parses the file
- If parsing is unsuccessful, the system displays an error message
- Otherwise, the program continues to the next configuration step or starts to run the publisher if there aren't more configurations

Id: PUBS-3

Title: Dataset field configuration

Description:

Before starting the Publisher:

- The user can choose a file that contains information about the dataset field configurations for the publisher node like the nodeId, namespaceIndex and AttributeId
- The program parses the file
- If parsing is unsuccessful, the displays an error message
- Otherwise, the program continues to the next configuration step or starts to run the publisher if there aren't more configurations

Id: PUBS-4

Title: Writer group configuration

Description:

Before starting the Publisher:

- The user can define in a config file the informations about the configuration of writer groups for the publisher node like:
 - The writer group name
 - The writer group enable flag
 - The writer group numeric identifier
 - The writer group publishing interval
- The program parses the file
- If parsing is unsuccessful, it displays an error message
- Otherwise, the program extracts the configuration for the writer group and continues to the next configuration step or starts to run the publisher if there are no more configurations

Id: PUBS-5

Title: Dataset writer configuration

Description:

After creating a PubSub connection, a writer group and a published dataset and before starting the server that hosts the publisher:

- The user can define in a config file the informations about a dataset writer like:
 - The name of the Dataset Writer
 - Dataset Writer integer id
 - The key frame number
 - The publishedDataset name linked to the datasetWriter
- The program parses the file
- If parsing is unsuccessful, the displays an error message
- Otherwise, the program continues to the next configuration step or starts to run the publisher if there aren't more configurations

Id: PUBS-6

Title: Certificate trust list configuration

Description:

Before starting the Publisher, the certificate trust list related to sks will be set to ensure a secure connection:

- The user has to configure the certificate trust-list in a specific directory
- The program load the certificates
- If the loading is unsuccessful, it's displayed an error message
- Otherwise, the program continues to the next configuration step or starts to run the publisher if there aren't more configurations

2.5.2. Subscriber

Id: SUBS-1

Title: Subscriber network configuration

Description:

Before starting Subscriber:

- The user can specify in a config file a set of network configuration like:
 - The IP of the network interface used to read data
 - The Port number on which the Subscriber Server will accept client communications
 - The UDP multicast IP and Port
 - The Publisher id type (string or integer) and Publisher id value from which the subscriber reads data
 - The Option to enable the connection
 - The Connection name
- System parses the file
- If parsing is unsuccessful, the program terminates and displays an error message
- Otherwise, the program continues to the next configuration step or starts to run the server if there is no more configuration to be done

Id: SUBS-2

Title: Reader Group Configuration

Description:

Before starting the Subscriber:

- The user can specify in a config file a set of configuration for the reader group like the reader group name
- The program parses the file
- If parsing is unsuccessful, it displays an error message
- Otherwise, the program returns a configuration object for the group configuration

Id: SUBS-3

Title: Dataset reader configuration

Description:

Before starting the Subscriber:

- The user can store the relevant configuration data for the dataset readers of the subscriber in a text-based file. In particular information like:
 - The dataset reader name
 - The dataset reader language
 - The dataset reader namespace index
 - The dataset reader publisher identifier
 - The dataset reader writer group identifier
 - The dataset writer identifier on the dataset reader
 - the dataset reader fields configurations
- The user can choose a file that contains information about dataset readers for the subscriber node
- The program parses the file
- If parsing is unsuccessful, it displays an error message
- Otherwise, the program returns a configuration object for a dataset reader

Id: SUBS-4

Title: Subscriber SKS reference configuration

Description:

Before starting the Subscriber:

- The user can store the relevant configuration data for the correct pulling of keys from the SKS in a text-based file. In particular information like:
 - The SKS IP address
 - The SKS Port Number
 - The Key Time Interval
 - The SKS private key repository name
- The user can choose a file that contains information about the SKS reference for the subscriber node
- The program parses the file
- If parsing is unsuccessful, it displays an error message
- Otherwise, the program returns a configuration object for a dataset reader

Id: SUBS-5

Title: Certificate trust list configuration

Description:

Before starting the Subscriber, the certificate trust list related to sks will be set to ensure a secure connection:

- The user has to configure the certificate trust-list in a specific directory
- The program load the certificates
- If the loading is unsuccessful, it's displayed an error message
- Otherwise, the program continues to the next configuration step or starts to run the subscriber if there aren't more configurations

2.5.3. SKS

Id: SKSS-1

Title: Subscriber network configuration

Description:

Before starting the server that hosts the SKS:

- The user can store the relevant configuration data for the SKS in a text-based file. Information like the server port.
- The program will parse the configuration file.
- If parsing is unsuccessful, the program terminates and displays an error message
- If parsing is successful, the program will proceed to the next configuration step or start the server if no further configuration is needed.

Id: SKSS-2

Title: SKS publisher configuration

Description:

Before starting the server that hosts the SKS:

- The user can choose a file that contains endpoint descriptions for all the publisher the sks needs to be responsible for
- System parses the file
- If parsing is unsuccessful, the program terminates and displays an error message
- Otherwise, it registers the endpoints as key push targets

Id: SKSS-3

Title: Security group configuration

Description:

Before starting the server that hosts the SKS:

- The user can choose a file that contains security group settings
- System parses the file
- If parsing is unsuccessful, the program displays an error message
- Otherwise, it creates a new security group using the settings in the file

Id: SKSS-4

Title: Subscriber Certificate trust list configuration

Description:

Before starting the Sks, the certificate trust list related to subscriber will be set to ensure a secure connection:

- The user has to configure the certificate trust-list in a specific directory
- The program load the certificates
- If the loading is unsuccessful, it's displayed an error message
- Otherwise, the program continues to the next configuration step or starts to run the sks if there aren't more configurations

Id: SKSS-5

Title: Publisher Certificate trust list configuration

Description:

Before starting the Sks, the certificate trust list related to publisher will be set to ensure a secure connection:

- The user has to configure the certificate trust-list in a specific directory
- The program load the certificates
- If the loading is unsuccessful, it's displayed an error message
- Otherwise, the program continues to the next configuration step or starts to run the sks if there aren't more configurations

Id: SKSS-6

Title: Recovery mechanism

Description:

After the server's reboot, the SKS needs to automatically recover so that encryption keys are available and operational continuity is maintained without manual intervention:

- The system checks if the database and encryption keys exist upon reboot.
- If the current encryption key exists, it is retrieved and used.
- If the current key is missing, a new encryption key and future keys are generated and stored.
- If the database does not exist, the system creates the necessary database and tables.
- If the database or tables cannot be created, the system returns an error.

2.5.4. Certificate & Private key generation tool

Id: CPKGS-1

Title: Certificate generation

Description:

The following steps have to be followed to generate certificate and private key:

- the user has to insert the following informations in the xml configuration file:
 - Subject list attributes as
 - Common Name
 - Organization Name
 - Organization Unit
 - Country
 - Alternative Subject list attributes as
 - DNS
 - URI
 - Expiration date
- System tries to parse the file
- The certificate and private key are generated and stored in a file

2.5.5. Round-trip time measuring tool

Id: RTTS-1

Title: Primary node network configuration

Description:

Before starting the primary node

- The user can specify network settings, such as ip address for the node, through a configuration file.
- System tries to parse the file
- If it does not manage to parse the file, an error is returned and the program terminates
- Otherwise, it continues to the next configuration step or starts running

Id: RTTS-2

Title: Primary node rounds configuration

Description:

Before starting the primary node

- The user can specify how many Round-trip rounds will be executed through a console command line argument.
- System tries to parse the file
- If it does not manage to parse the file, an error is returned and the program terminates
- Otherwise, it continues to the next configuration step or starts running the specified number of round trips

Id: RTTS-3

Title: SKS customization

Description:

Before starting the primary or intermediate nodes

- The user can specify if the SKS will be enabled through a console command line argument.
- System tries to parse the file
- If it does not manage to parse the file, an error is returned and the program terminates
- Otherwise, it continues to the next configuration step or starts running with or without an SKS depending on the configuration

Id: RTTS-4

Title: Data collection

Description:

Before running RTT

- The user can specify the format in which the data will be logged.
- During the experiment, the RTT data is collected
- After the experiment is finished, the data is logged in the format of user's choosing

Id: RTTS-5

Title: Intermediate node network configuration

Description:

Before starting the intermediate node

- The user can specify network settings, such as ip address for the node, through a configuration file.

- System tries to parse the file
- If it does not manage to parse the file, an error is returned and the program terminates
- Otherwise, it continues to the next configuration step or starts running

Id: RTTS-6

Title: Intermediate node SKS configuration

Description:

Before starting the intermediate node

- The user can specify if the SKS will be enabled through a console command line argument.
- System tries to parse the file
- If it does not manage to parse the file, an error is returned and the program terminates
- Otherwise, it continues to the next configuration step or starts running with or without an SKS depending on the configuration

2.6. Mapping table between user stories and requirements

User story ID	List of requirements IDs
PUBS-1	PUB-1 PUB-1.1 PUB-1.2 PUB-1.3 PUB-1.4 PUB-1.5 PUB-1.6
PUBS-2	PUB-2 PUB-2.1
PUBS-3	PUB-3 PUB-3.1 PUB-3.2 PUB-3.3
PUBS-4	PUB-4 PUB-4.1 PUB-4.2 PUB-4.3 PUB-4.4
PUBS-5	PUB-5 PUB-5.1 PUB-5.2 PUB-5.3 PUB-5.4
PUBS-6	PUB-6 PUB-6.1 PUB-6.2
SUBS-1	SUB-1 SUB-1.1 SUB-1.2 SUB-1.3 SUB-1.4 SUB-1.5 SUB-1.6
SUBS-2	SUB-2 SUB-2.1
SUBS-3	SUB-3 SUB-3.1 SUB-3.2 SUB-3.3 SUB-3.4 SUB-3.5

User story ID	List of requirements IDs
	SUB-3.6 SUB-3.7 SUB-3.7.1 SUB-3.7.2 SUB-3.7.3 SUB-3.7.4
SUBS-4	SUB-4 SUB-4.1 SUB-4.2 SUB-4.3 SUB-4.4
SUBS-5	SUB-5 SUB-5.1 SUB-5.2
SKSS-1	SKS-1 SKS-1.1
SKSS-2	SKS-2 SKS-2.1
SKSS-3	SKS-3 SKS-3.2 SKS-3.3 SKS-3.4 SKS-3.5 SKS-3.6
SKSS-4	SKS-4 SKS-4.1 SKS-4.2
SKSS-5	SKS-5 SKS-5.1 SKS-5.2
SKSS-6	SKS-6 SKS-7
CPKGS-1	CPKG-1
RTTS-1	RTT-1
RTTS-2	RTT-3
RTTS-3	RTT-4
RTTS-4	RTT-5

2.7. Graphical User Interface

This project excludes the development of a GUI. The choice to forego a graphical interface reflects the customer's and final users' needs, as well as the nature of the pre-existing technology itself.

As stated by the client, the end-users are to be developers, industry specialists and engineers comfortable with XML syntax and the OPC UA technology, who will benefit from this solution to facilitate their work as technical users of the open62541 stack.

Indeed, this project can be seen as an extension of the already existing functionalities of the open62541 stack, and as such, it is **used as a library** by developers that need to set up communication infrastructures between industrial components.

ARB